

# CarePulse Twin

**Real-Time Machine Learning Framework  
for Patient Deterioration Prediction**

**PROJECT REPORT**

Submitted by

**K. Jitin Venkata Sai**

International Institute of Information Technology Bangalore

## Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1 Abstract</b>                                            | <b>4</b>  |
| <b>2 Introduction</b>                                        | <b>4</b>  |
| <b>3 Project Objectives</b>                                  | <b>4</b>  |
| <b>4 System Architecture</b>                                 | <b>5</b>  |
| 4.1 Architectural Components . . . . .                       | 6         |
| <b>5 Dataset</b>                                             | <b>6</b>  |
| 5.1 Dataset Source . . . . .                                 | 6         |
| 5.2 Why the eICU Dataset? . . . . .                          | 6         |
| <b>6 Raw Dataset Structure</b>                               | <b>6</b>  |
| 6.1 vitalPeriodic.csv.gz . . . . .                           | 6         |
| 6.2 vitalAperiodic.csv.gz . . . . .                          | 7         |
| 6.3 patient.csv.gz . . . . .                                 | 7         |
| <b>7 Feature Selection</b>                                   | <b>7</b>  |
| 7.1 Why These Features? . . . . .                            | 8         |
| <b>8 Dataset Exploration</b>                                 | <b>8</b>  |
| <b>9 Data Preprocessing</b>                                  | <b>8</b>  |
| 9.1 Step 1: Load Raw Periodic Vitals . . . . .               | 8         |
| 9.2 Step 2: Convert Observation Offset to Hour . . . . .     | 9         |
| 9.3 Step 3: Group Observations by Patient and Hour . . . . . | 9         |
| 9.4 Step 4: Produce the Hourly Dataset . . . . .             | 9         |
| 9.5 Processed Dataset Size . . . . .                         | 9         |
| 9.6 Processed Data Example . . . . .                         | 10        |
| <b>10 Physiological Range Validation</b>                     | <b>10</b> |
| <b>11 Abnormality Definition</b>                             | <b>10</b> |
| <b>12 Temporal Sequence Construction</b>                     | <b>10</b> |
| 12.1 Why 24 Hours? . . . . .                                 | 11        |
| <b>13 Prediction Target</b>                                  | <b>11</b> |
| <b>14 Feature Normalization</b>                              | <b>11</b> |
| 14.1 Training Statistics . . . . .                           | 12        |
| <b>15 Machine Learning Model</b>                             | <b>12</b> |
| 15.1 Why LSTM? . . . . .                                     | 12        |
| 15.2 Model Input . . . . .                                   | 13        |
| 15.3 Model Output . . . . .                                  | 13        |
| <b>16 Model Training</b>                                     | <b>13</b> |
| 16.1 Loss Function . . . . .                                 | 13        |
| 16.2 Optimizer . . . . .                                     | 13        |
| 16.3 Gradient Clipping . . . . .                             | 13        |
| <b>17 Handling Class Imbalance</b>                           | <b>14</b> |
| <b>18 Training, Validation and Test Sets</b>                 | <b>14</b> |
| <b>19 Prediction Threshold Selection</b>                     | <b>14</b> |
| <b>20 Model Evaluation</b>                                   | <b>14</b> |
| <b>21 Model Checkpoint</b>                                   | <b>15</b> |

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>22 Risk Engine</b>                            | <b>15</b> |
| 22.1 Patient Window . . . . .                    | 15        |
| <b>23 Risk Score</b>                             | <b>16</b> |
| <b>24 Risk Severity</b>                          | <b>16</b> |
| <b>25 Risk Explainability</b>                    | <b>16</b> |
| <b>26 FastAPI Backend</b>                        | <b>16</b> |
| 26.1 Health Endpoint . . . . .                   | 17        |
| 26.2 Ingestion Endpoint . . . . .                | 17        |
| <b>27 Apache Kafka Streaming Layer</b>           | <b>17</b> |
| 27.1 Why Kafka? . . . . .                        | 17        |
| <b>28 Redis</b>                                  | <b>18</b> |
| <b>29 PostgreSQL</b>                             | <b>18</b> |
| <b>30 Historical Replay System</b>               | <b>19</b> |
| 30.1 Why Replay Is Important . . . . .           | 19        |
| <b>31 Example Patient Demonstration</b>          | <b>19</b> |
| <b>32 Example of Detected Deterioration</b>      | <b>20</b> |
| <b>33 Software and Libraries</b>                 | <b>20</b> |
| <b>34 Why These Technologies Were Selected</b>   | <b>21</b> |
| 34.1 Python . . . . .                            | 21        |
| 34.2 Pandas . . . . .                            | 21        |
| 34.3 NumPy . . . . .                             | 21        |
| 34.4 PyTorch . . . . .                           | 21        |
| 34.5 scikit-learn . . . . .                      | 22        |
| 34.6 FastAPI . . . . .                           | 22        |
| 34.7 Kafka . . . . .                             | 22        |
| 34.8 Redis . . . . .                             | 22        |
| 34.9 PostgreSQL . . . . .                        | 22        |
| 34.10 Docker Compose . . . . .                   | 23        |
| <b>35 Complete End-to-End Data Flow</b>          | <b>23</b> |
| 35.1 Stage 1: Historical Data . . . . .          | 23        |
| 35.2 Stage 2: Preprocessing . . . . .            | 23        |
| 35.3 Stage 3: Sequence Construction . . . . .    | 23        |
| 35.4 Stage 4: Training . . . . .                 | 23        |
| 35.5 Stage 5: Model Deployment . . . . .         | 23        |
| 35.6 Stage 6: Data Ingestion . . . . .           | 23        |
| 35.7 Stage 7: Validation . . . . .               | 23        |
| 35.8 Stage 8: Kafka . . . . .                    | 23        |
| 35.9 Stage 9: Inference . . . . .                | 23        |
| 35.10 Stage 10: Model Prediction . . . . .       | 23        |
| 35.11 Stage 11: Explainability . . . . .         | 24        |
| 35.12 Stage 12: State Storage . . . . .          | 24        |
| 35.13 Stage 13: Dashboard . . . . .              | 24        |
| <b>36 Complete Demonstration Procedure</b>       | <b>24</b> |
| 36.1 Step 1: Enter the Project . . . . .         | 24        |
| 36.2 Step 2: Start Infrastructure . . . . .      | 24        |
| 36.3 Step 3: Verify Kafka Topic . . . . .        | 24        |
| 36.4 Step 4: Start Backend . . . . .             | 24        |
| 36.5 Step 5: Check Health . . . . .              | 24        |
| 36.6 Step 6: Inspect Patients . . . . .          | 24        |
| 36.7 Step 7: Replay Historical Patient . . . . . | 25        |
| 36.8 Step 8: Verify Redis State . . . . .        | 25        |

|                                            |           |
|--------------------------------------------|-----------|
| 36.9 Step 9: Verify PostgreSQL . . . . .   | 25        |
| 36.10 Step 10: Inspect Alerts . . . . .    | 25        |
| <b>37 How to Explain the Demo</b>          | <b>25</b> |
| <b>38 Important Design Distinction</b>     | <b>25</b> |
| 38.1 Machine Learning Prediction . . . . . | 25        |
| 38.2 Rule-Based Explanation . . . . .      | 25        |
| <b>39 Current Limitations</b>              | <b>26</b> |
| <b>40 Future Work</b>                      | <b>26</b> |
| <b>41 Project Summary</b>                  | <b>26</b> |
| <b>A Project Directory Structure</b>       | <b>28</b> |
| <b>B Key Commands</b>                      | <b>28</b> |
| B.1 Prepare Dataset . . . . .              | 28        |
| B.2 Train Model . . . . .                  | 28        |
| B.3 Start Backend . . . . .                | 28        |
| B.4 Replay Patient . . . . .               | 28        |
| B.5 Check Health . . . . .                 | 29        |
| B.6 Check Current Patient State . . . . .  | 29        |
| B.7 Check Alerts . . . . .                 | 29        |

## 1. Abstract

CarePulse Twin is a real-time patient monitoring and deterioration prediction system designed around physiological vital-sign data. The system combines historical ICU data, machine learning, stream processing, a persistent data layer, and a real-time API to create a continuously updated computational representation of a patient's physiological state.

The project uses the eICU Collaborative Research Database as the primary source of physiological observations. The raw data contains periodic vital measurements such as heart rate, oxygen saturation, blood pressure, and respiratory rate. These measurements are converted into an hourly representation and subsequently transformed into fixed-length temporal sequences.

A Long Short-Term Memory (LSTM) neural network is trained to use the previous 24 hours of physiological information to predict whether a qualifying vital abnormality will occur during the following 6 hours.

The trained model is integrated into a streaming architecture. Patient observations are submitted through a FastAPI service, published to Apache Kafka, consumed by the inference engine, processed using the LSTM model and rule-based explainability logic, and stored in Redis and PostgreSQL.

The final system therefore combines:

- historical ICU physiological data,
- temporal preprocessing,
- machine learning based risk prediction,
- rule-based safety screening,
- Kafka-based streaming,
- Redis-based current-state storage,
- PostgreSQL persistent storage,
- FastAPI REST and WebSocket interfaces,
- and a browser-based monitoring dashboard.

## 2. Introduction

Continuous monitoring of physiological variables is an important component of intensive care. Patients generate a stream of measurements such as heart rate, oxygen saturation, blood pressure, temperature, and respiratory rate.

The difficulty is not simply storing these measurements. The system must determine whether the patient's current physiological trajectory indicates an increased risk of deterioration.

CarePulse Twin addresses this problem by maintaining a temporal representation of a patient and continuously updating a deterioration risk score.

The central idea is:

Instead of evaluating a vital sign independently, CarePulse Twin evaluates a temporal window of physiological observations and uses that information to estimate the probability of deterioration.

For every patient, the system maintains a sequence of recent observations:

$$X = [x_{t-23}, x_{t-22}, \dots, x_{t-1}, x_t]$$

where every  $x_t$  contains six physiological features.

The LSTM receives this sequence and produces a probability:

$$p_t = P(Y_{t+1:t+6} = 1 \mid X_{t-23:t})$$

where the target represents whether a qualifying abnormality occurs during the following six hours.

## 3. Project Objectives

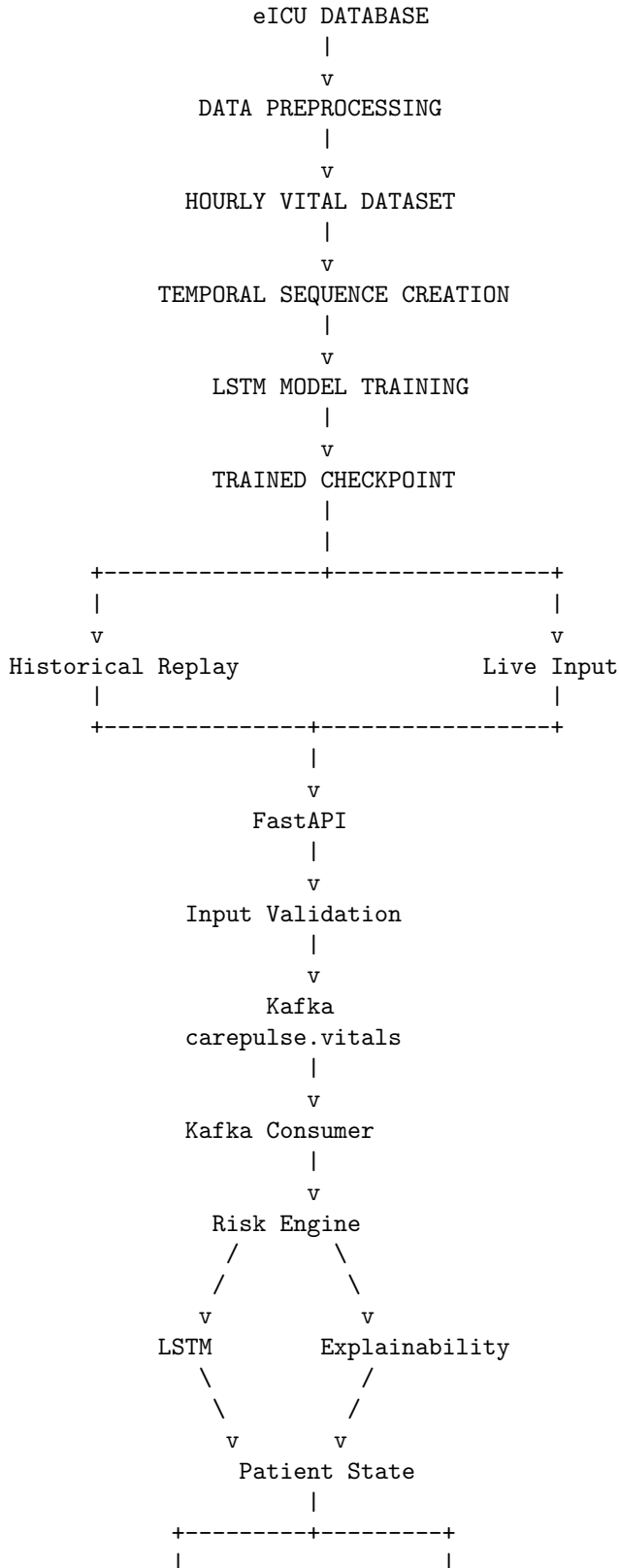
The main objectives of CarePulse Twin are:

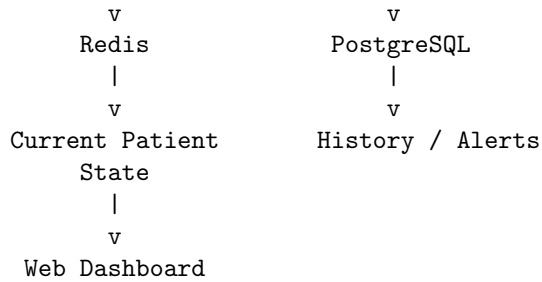
1. Process physiological ICU data into a clean temporal dataset.
2. Represent patient physiology using multiple vital signs rather than relying on a single measurement.
3. Construct fixed-length temporal windows for machine learning.
4. Predict whether a qualifying vital abnormality will occur in the next six hours.
5. Produce a continuous risk probability between 0 and 1.
6. Convert the probability into clinically interpretable severity categories.

7. Detect obviously invalid physiological inputs before they enter the inference pipeline.
8. Provide human-readable explanations for elevated risk.
9. Support real-time ingestion through a REST API.
10. Use Apache Kafka for asynchronous event streaming.
11. Store the latest patient state in Redis for low-latency access.
12. Store historical vitals, alerts, and audit information in PostgreSQL.
13. Provide a browser-based interface for monitoring a patient.

## 4. System Architecture

The complete system can be represented as follows:





#### 4.1 Architectural Components

The architecture consists of the following major components.

1. **eICU Dataset:** Provides historical ICU physiological observations.
2. **Preprocessing Pipeline:** Converts irregular raw measurements into an hourly, machine-learning-ready representation.
3. **LSTM Model:** Learns temporal relationships between physiological variables.
4. **FastAPI Backend:** Provides REST endpoints and WebSocket communication.
5. **Apache Kafka:** Acts as the event-streaming layer between ingestion and inference.
6. **Risk Engine:** Maintains the latest 24-hour window and invokes the trained model.
7. **Redis:** Stores the most recent computed state of a patient.
8. **PostgreSQL:** Stores persistent vitals, alerts, and audit records.
9. **Replay Engine:** Reconstructs historical patient streams from the eICU dataset.
10. **Frontend:** Displays current risk, physiological variables, and explanations.

### 5. Dataset

#### 5.1 Dataset Source

The project uses physiological data derived from the **eICU Collaborative Research Database**.

The local dataset directory used by the project is:

```
data/eicu/
```

The available files include:

```
vitalPeriodic.csv.gz
vitalAperiodic.csv.gz
patient.csv.gz
```

#### 5.2 Why the eICU Dataset?

The project requires longitudinal physiological observations rather than isolated samples.

The eICU dataset is suitable because it contains ICU patient observations collected over time. This allows the project to construct temporal sequences and study physiological trajectories.

The important aspect for CarePulse Twin is therefore not merely the number of patients, but the existence of repeated measurements for each ICU stay.

### 6. Raw Dataset Structure

#### 6.1 vitalPeriodic.csv.gz

The main physiological source used for model preprocessing is:

```
data/eicu/vitalPeriodic.csv.gz
```

The relevant columns used by the preprocessing pipeline are:

Table 1: Relevant periodic vital columns

| Raw Column        | Meaning                  | Model Feature    |
|-------------------|--------------------------|------------------|
| patientunitstayid | ICU stay identifier      | Patient ID       |
| observationoffset | Time from ICU admission  | Hour/time        |
| heartrate         | Heart rate               | heart_rate       |
| sao2              | Oxygen saturation        | spo2             |
| systemicsystolic  | Systolic blood pressure  | systolic_bp      |
| systemicdiastolic | Diastolic blood pressure | diastolic_bp     |
| temperature       | Temperature              | temperature      |
| respiration       | Respiratory rate         | respiratory_rate |

## 6.2 vitalAperiodic.csv.gz

The project also contains:

`data/eicu/vitalAperiodic.csv.gz`

The inspected columns include:

```
vitalaperiodicid
patientunitstayid
observationoffset
noninvasivesystolic
noninvasivediastolic
noninvasivemean
paop
cardiacoutput
cardiacinput
svr
svri
pvr
pvri
```

These measurements are aperiodic/non-periodic physiological measurements. The current six-feature LSTM pipeline, however, is constructed from the selected periodic vital variables.

## 6.3 patient.csv.gz

The dataset directory also contains:

`data/eicu/patient.csv.gz`

This file provides patient/ICU-stay level information. The current prediction pipeline primarily uses the physiological time-series tables for the six selected model features.

## 7. Feature Selection

The model uses exactly six physiological variables:

1. Heart rate
2. SpO<sub>2</sub>
3. Systolic blood pressure
4. Diastolic blood pressure
5. Temperature
6. Respiratory rate

Therefore the feature vector at time  $t$  is:

$$x_t = [HR_t, SpO_{2t}, SBP_t, DBP_t, Temp_t, RR_t]$$

and has dimensionality:

$$x_t \in \mathbb{R}^6$$



### 7.1 Why These Features?

These variables provide complementary information about cardiovascular, respiratory, and general physiological state.

Table 2: Selected physiological features

| Feature          | Purpose                                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Heart rate       | Represents cardiovascular rate and may increase during physiological stress.                                                    |
| SpO <sub>2</sub> | Represents blood oxygen saturation and is particularly important for respiratory deterioration.                                 |
| Systolic BP      | Provides information about arterial pressure and cardiovascular status.                                                         |
| Diastolic BP     | Provides complementary blood pressure information.                                                                              |
| Temperature      | Can provide information about fever or other physiological changes.                                                             |
| Respiratory rate | Provides a direct indicator of respiratory stress and is particularly useful for identifying abnormal respiratory trajectories. |

## 8. Dataset Exploration

For patient 521456, the raw periodic vital data contained:

13150

observations.

The availability of the selected raw variables was:

Table 3: Raw feature availability for patient 521456

| Feature          | Available | Missing (%) |
|------------------|-----------|-------------|
| Heart rate       | 13109     | 0.31        |
| SpO <sub>2</sub> | 12990     | 1.22        |
| Systolic BP      | 6046      | 54.02       |
| Diastolic BP     | 6046      | 54.02       |
| Temperature      | 0         | 100.00      |
| Respiratory rate | 12022     | 8.58        |

This inspection demonstrates an important characteristic of ICU data: different physiological measurements are not necessarily recorded at the same frequency.

For example, blood pressure is substantially less complete than heart rate, while temperature was unavailable for this particular patient in the inspected periodic table.

## 9. Data Preprocessing

The raw eICU measurements cannot be directly provided to the LSTM because they are timestamped observations with varying measurement frequency.

The preprocessing pipeline therefore converts the raw measurements into a regular hourly representation.

The implemented preprocessing script is:

```
data/prepare.py
```

### 9.1 Step 1: Load Raw Periodic Vitals

The relevant columns are loaded from:

```
vitalPeriodic.csv.gz
```

The preprocessing selects:

```

patientunitstayid
observationoffset
temperature
sao2
heartrate
respiration
systemicsystolic
systemicdiastolic

```

## 9.2 Step 2: Convert Observation Offset to Hour

The eICU periodic table contains:

```
observationoffset
```

which represents the observation offset from ICU admission.

The pipeline converts this temporal information into an hourly representation.

This is important because the LSTM requires a consistent temporal grid.

## 9.3 Step 3: Group Observations by Patient and Hour

Measurements are grouped using:

$$(patientunitstayid, hour)$$

For each patient-hour, the pipeline computes the median of the available observations for the six selected features.

Conceptually:

$$x_{patient, hour} = median(x_{patient, hour, 1}, x_{patient, hour, 2}, \dots)$$

The median is useful because it reduces the effect of isolated measurement spikes.

## 9.4 Step 4: Produce the Hourly Dataset

The resulting dataset is:

```
data/processed/hourly_vitals.csv
```

The columns are:

```

patientunitstayid
hour
temperature
spo2
heart_rate
respiratory_rate
systolic_bp
diastolic_bp
abnormal
label

```

## 9.5 Processed Dataset Size

The processed dataset contains:

139917

rows and:

2380

patients.

Thus, the preprocessing converts a large collection of irregular physiological measurements into a structured temporal dataset.

## 9.6 Processed Data Example

An example from the processed dataset is:

```
patientunitstayid , hour , temperature , spo2 , heart_rate ,
respiratory_rate , systolic_bp , diastolic_bp , abnormal , label
```

```
141764 , 0 , 37.2 , 97.0 , 119.0 , 19.0 , 121.0 , 64.0 , 0 , 0
141764 , 1 , 37.2 , 96.0 , 105.0 , 19.0 , 173.5 , 88.5 , 0 , 0
141764 , 2 , 37.2 , 99.5 , 95.0 , 19.0 , 173.0 , 106.0 , 0 , 0
```

## 10. Physiological Range Validation

The preprocessing and ingestion pipeline defines acceptable physiological ranges.

The ranges are:

Table 4: Input validation ranges

| Feature          | Minimum | Maximum |
|------------------|---------|---------|
| Heart rate       | 20      | 250     |
| SpO <sub>2</sub> | 50      | 100     |
| Systolic BP      | 40      | 300     |
| Diastolic BP     | 20      | 200     |
| Temperature      | 25      | 45      |
| Respiratory rate | 2       | 80      |

Values outside these ranges are flagged as invalid.

Additional clinical-style flags currently include:

- SpO<sub>2</sub> < 90
- Respiratory rate > 30

The purpose of this layer is not to replace the machine learning model. It provides an input-quality and safety screening layer before the observation enters the streaming inference pipeline.

## 11. Abnormality Definition

The preprocessing pipeline creates an **abnormal** indicator based on qualifying abnormal vital ranges.

This provides a rule-based representation of whether the current hour contains a qualifying abnormal vital.

The machine learning target is different.

The model target is:

**Any qualifying vital abnormality in the following 6 hours.**

This distinction is important.

The current abnormality indicator describes the current observation, while the model target represents a future event.

## 12. Temporal Sequence Construction

The LSTM does not receive one row at a time.

Instead, it receives a sequence of the previous 24 hourly observations.

The sequence length is:

$$L = 24$$

Each observation contains six features.

Therefore the model input has shape:

$$24 \times 6$$

For a patient at hour  $t$ , the input is:

$$X_t = \begin{bmatrix} x_{t-23} \\ x_{t-22} \\ \vdots \\ x_{t-1} \\ x_t \end{bmatrix}$$

where:

$$x_t \in \mathbb{R}^6$$

Therefore:

$$X_t \in \mathbb{R}^{24 \times 6}$$

### 12.1 Why 24 Hours?

A 24-hour sequence allows the model to observe approximately one day of physiological history.

This enables the LSTM to learn patterns such as:

- persistent elevation of respiratory rate,
- decreasing oxygen saturation,
- changes in heart rate,
- combined changes across multiple vital signs,
- and temporal trends rather than isolated measurements.

## 13. Prediction Target

The target is defined over the following six hours.

For an input sequence ending at time  $t$ :

$$X_t = [x_{t-23}, \dots, x_t]$$

the model predicts:

$$Y_t = \begin{cases} 1 & \text{if a qualifying abnormality occurs in } t+1, \dots, t+6 \\ 0 & \text{otherwise} \end{cases}$$

Thus the model performs a future deterioration prediction task rather than simply detecting an abnormality that has already occurred.

## 14. Feature Normalization

The six features have very different numerical scales.

For example:

$$HR \approx 80$$

while:

$$SpO_2 \approx 96$$

and blood pressure can be above 100.

To prevent features with larger numerical scales from dominating training, standardization is applied:

$$z = \frac{x - \mu}{\sigma}$$

where:

- $\mu$  is the feature mean,
- $\sigma$  is the feature standard deviation.

The checkpoint stores the normalization statistics so that inference uses the exact same transformation as training.

### 14.1 Training Statistics

The checkpoint contains the following means:

```
[86.330238 ,  
 96.006576 ,  
 124.515579 ,  
 65.540909 ,  
 36.959465 ,  
 20.360004]
```

and standard deviations:

```
[17.227689 ,  
 3.355853 ,  
 23.746471 ,  
 14.764313 ,  
 0.705030 ,  
 5.818191]
```

The ordering corresponds to:

```
heart_rate  
spo2  
systolic_bp  
diastolic_bp  
temperature  
respiratory_rate
```

## 15. Machine Learning Model

The project uses a Long Short-Term Memory network.

The implementation is contained in:

```
ml/model.py
```

and training is implemented in:

```
ml/train.py
```

### 15.1 Why LSTM?

Physiological deterioration is inherently temporal.

For example, the following two situations are different:

$$RR = 30$$

at one isolated point, versus:

$$RR : 18 \rightarrow 22 \rightarrow 26 \rightarrow 30 \rightarrow 34$$

over several hours.

The second pattern contains temporal information.

LSTMs are designed to learn dependencies across sequential observations. Their recurrent memory mechanism allows information from earlier time steps to influence predictions at later time steps.

## 15.2 Model Input

The model receives:

$$(batch\_size, 24, 6)$$

where:

- 24 is the sequence length,
- 6 is the number of physiological features.

## 15.3 Model Output

The model produces a scalar logit.

A sigmoid function converts the logit into a probability:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Therefore:

$$0 \leq p \leq 1$$

The probability represents the model's estimated likelihood of the defined deterioration target.

## 16. Model Training

Training is implemented using PyTorch.

The major training components are:

- PyTorch tensors,
- DataLoader,
- Adam optimizer,
- BCEWithLogitsLoss,
- gradient clipping,
- validation evaluation,
- threshold optimization.

### 16.1 Loss Function

Binary cross entropy with logits is used:

$$\mathcal{L} = -[y \log(p) + (1 - y) \log(1 - p)]$$

The implementation uses:

```
nn.BCEWithLogitsLoss()
```

Using the logits version combines numerical stability with the binary classification objective.

### 16.2 Optimizer

The model uses Adam:

```
torch.optim.Adam(
    model.parameters(),
    lr=1e-3
)
```

Adam is used because it provides adaptive parameter updates and is well suited to neural-network optimization.

### 16.3 Gradient Clipping

The implementation clips gradients:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

This limits excessively large gradients and improves training stability for recurrent networks.

## 17. Handling Class Imbalance

The deterioration target is imbalanced.

The training code therefore creates a balanced training subset.

The procedure is:

1. Identify positive samples.
2. Identify negative samples.
3. Randomly select at most twice as many negative samples as positive samples.
4. Combine the positive and selected negative examples.
5. Shuffle the resulting training set.

The implementation uses a fixed random seed:

```
np.random.default_rng(11)
```

This improves reproducibility.

## 18. Training, Validation and Test Sets

The processed sequence dataset contains separate:

```
X_train , y_train
X_val , y_val
X_test , y_test
```

The validation set is used to select the best model according to validation ROC-AUC.

The test set is used only after model selection for final evaluation.

## 19. Prediction Threshold Selection

The default probability threshold for binary classification is often:

$$p \geq 0.5$$

However, the project does not simply assume 0.5.

During validation, precision and recall are calculated for different thresholds.

The F1 score is:

$$F_1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

The threshold corresponding to the best validation F1 score is stored in the model checkpoint.

For the reproduced model, the selected threshold was approximately:

0.3266

The checkpoint therefore stores the threshold alongside the model weights and normalization statistics.

## 20. Model Evaluation

The reproduced model achieved the following test results:

Table 5: Reproduced model test performance

| Metric                | Value  |
|-----------------------|--------|
| ROC-AUC               | 0.8438 |
| PR-AUC                | 0.6589 |
| F1 Score              | 0.6021 |
| Precision             | 0.5331 |
| Recall                | 0.6915 |
| Test samples          | 14691  |
| Positive test samples | 2956   |
| Threshold             | 0.3266 |

The confusion matrix was:

$$\begin{bmatrix} 9945 & 1790 \\ 912 & 2044 \end{bmatrix}$$

where the rows represent the true class and the columns represent the predicted class.

## 21. Model Checkpoint

The trained model is stored as:

```
ml/artifacts/carepulse_lstm.pt
```

The checkpoint contains:

```
state_dict
mean
std
seq_len
feature_names
threshold
target
```

This is important because deploying a model requires more than just the neural-network weights.

The inference engine must also know:

- how the input was normalized,
- how many time steps are required,
- the ordering of features,
- and which threshold was selected.

## 22. Risk Engine

The deployed inference logic is implemented in:

```
backend/core.py
```

The central component is:

```
RiskEngine
```

For every patient, the engine maintains:

```
deque(maxlen=24)
```

Therefore every patient has an independent recent temporal window.

### 22.1 Patient Window

When a new vital arrives:

$$x_t$$

is appended to that patient's deque.

Once 24 observations are available:

$$|W| = 24$$

the LSTM is activated.

Before the complete window exists, the deployed system can use a rule-based fallback score.



## 23. Risk Score

When the 24-hour window is available:

$$X = \frac{W - \mu}{\sigma}$$

is passed to the LSTM.

The model produces:

$$p = \sigma(f_{\theta}(X))$$

where  $f_{\theta}$  is the trained LSTM.

The resulting value is the current risk score.

## 24. Risk Severity

The risk score is converted into four severity levels.

Table 6: Risk severity thresholds

| Risk                 | Severity | Meaning                    |
|----------------------|----------|----------------------------|
| $r < 0.40$           | LOW      | Low current predicted risk |
| $0.40 \leq r < 0.65$ | MEDIUM   | Moderate predicted risk    |
| $0.65 \leq r < 0.85$ | HIGH     | High predicted risk        |
| $r \geq 0.85$        | CRITICAL | Very high predicted risk   |

## 25. Risk Explainability

A major design goal is that the system should not only output:

$$Risk = 0.81$$

but should also provide an explanation.

The system checks several interpretable factors.

Examples include:

- Low SpO<sub>2</sub>.
- Elevated respiratory rate.
- Elevated heart rate.
- Elevated temperature.
- Decline in SpO<sub>2</sub> over the recent window.
- Increasing respiratory rate.
- Increasing heart rate.

For example, the system may produce:

```
respiratory rate is 26.5/min
respiratory rate has an upward trend
```

This creates a human-readable connection between the model output and the patient's recent physiological trajectory.

## 26. FastAPI Backend

The backend is implemented using FastAPI.

The primary backend file is:

```
backend/main.py
```

The service runs using Uvicorn.

Example:

```
python -m uvicorn backend.main:app \
    --host 0.0.0.0 \
    --port 8001
```

### 26.1 Health Endpoint

The backend provides:

GET /health

Example response:

```
{
  "status": "ok",
  "model_available": true,
  "kafka_topic": "carepulse.vitals"
}
```

### 26.2 Ingestion Endpoint

The primary ingestion endpoint is:

POST /api/v1/ingest

A request contains:

```
patient_id
ts
heart_rate
spo2
systolic_bp
diastolic_bp
temperature
respiratory_rate
```

The backend validates the values before publishing them to Kafka.

## 27. Apache Kafka Streaming Layer

The Kafka topic used by the project is:

carepulse.vitals

Kafka is used to decouple data ingestion from inference.

The architecture is:

```
Client
|
v
FastAPI
|
v
Kafka Producer
|
v
carepulse.vitals
|
v
Kafka Consumer
|
v
Risk Engine
```

### 27.1 Why Kafka?

Without Kafka, the API would need to perform all inference work synchronously.

With Kafka:

- ingestion is separated from processing,

- observations become events,
- inference can consume events asynchronously,
- the architecture can later scale to multiple consumers.

The Kafka consumer group is:

```
carepulse—inference
```

## 28. Redis

Redis stores the latest computed state of each patient.

The key format is:

```
state:<patient_id>
```

For example:

```
state:521456
```

An example stored state is:

```
{
  "patient_id": "521456",
  "risk": 0.0725,
  "severity": "LOW",
  "window": 24,
  "model_available": true,
  "vitals": {
    "heart_rate": 79.0,
    "spo2": 100.0,
    "systolic_bp": 130.0,
    "diastolic_bp": 53.0,
    "temperature": 37.2,
    "respiratory_rate": 20.0
  }
}
```

Redis is appropriate for this purpose because the dashboard needs fast access to the current patient state.

## 29. PostgreSQL

PostgreSQL provides persistent storage.

The system stores:

- patient vitals,
- alerts,
- audit events.

The vital record contains:

```
patient_id
ts
heart_rate
spo2
systolic_bp
diastolic_bp
temperature
respiratory_rate
anomaly_flags
```

The alert table contains information such as:

```
patient_id
ts
severity
risk
factors
message
```

This creates a distinction between:

**Redis = current state**

and:

**PostgreSQL = persistent history**

### 30. Historical Replay System

Real-time patient data is not always available during development.

Therefore the project contains a replay mechanism:

`replay/replay.py`

The replay engine reads:

`data/processed/hourly_vitals.csv`

for a selected patient.

For example:

```
python -m replay.replay \
    --data data/processed/hourly_vitals.csv \
    --patient-id 521456 \
    --api http://localhost:8001 \
    --speed 3600
```

The replay engine reconstructs the historical sequence and sends each observation to the same production ingestion endpoint:

*Replay → FastAPI → Kafka → Inference → Redis/PostgreSQL*

#### 30.1 Why Replay Is Important

Replay allows the complete streaming pipeline to be tested without requiring a live hospital data feed.

It therefore acts as a bridge between:

- offline historical data, and
- online real-time inference.

### 31. Example Patient Demonstration

Patient:

521456

contains:

1105

hourly observations in the processed dataset.

The final state observed after replay was:

Table 7: Example final patient state

| Variable         | Value           |
|------------------|-----------------|
| Patient ID       | 521456          |
| Heart rate       | 79 bpm          |
| SpO <sub>2</sub> | 100%            |
| Systolic BP      | 130 mmHg        |
| Diastolic BP     | 53 mmHg         |
| Temperature      | 37.2°C          |
| Respiratory rate | 20/min          |
| Risk             | 0.0725          |
| Severity         | LOW             |
| Window           | 24 observations |
| Model active     | Yes             |

The explanation at that point was:

No dominant abnormal trend detected

### 32. Example of Detected Deterioration

During replay of patient 521456, the system generated elevated-risk states.

For example, one alert contained:

Risk: 0.8939

Severity: CRITICAL

respiratory rate is 34.0/min;  
 SpO<sub>2</sub> declined from 100.0% to 96.5%;  
 respiratory rate has an upward trend

This demonstrates the complete chain:

Physiological observations → Temporal window → LSTM risk → Severity → Explanation → Alert

### 33. Software and Libraries

The project combines several technologies.

Table 8: Technology stack

| Technology          | Purpose                                                                                             |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Python              | Primary programming language for preprocessing, machine learning, backend services, and replay.     |
| Pandas              | Reading, filtering, grouping, aggregating, and manipulating the eICU tabular data.                  |
| NumPy               | Numerical arrays, normalization, sampling, and efficient numerical operations.                      |
| PyTorch             | Building, training, saving, and running the LSTM model.                                             |
| scikit-learn        | Computing ROC-AUC, PR-AUC, F1, precision, recall, precision-recall curves, and confusion matrices.  |
| FastAPI             | Building the REST API and WebSocket interface.                                                      |
| Uvicorn             | ASGI server used to run the FastAPI application.                                                    |
| Apache Kafka        | Asynchronous event streaming between ingestion and inference.                                       |
| aiokafka            | Python asynchronous Kafka producer and consumer implementation.                                     |
| Redis               | Low-latency storage for the latest patient state.                                                   |
| PostgreSQL          | Persistent storage for vitals, alerts, and audit information.                                       |
| SQLAlchemy          | Database interaction from the FastAPI backend.                                                      |
| httpx               | HTTP client used by the replay engine to send observations to the backend.                          |
| Docker Compose      | Running infrastructure services such as Kafka, Redis, and PostgreSQL in a reproducible environment. |
| HTML/CSS/JavaScript | Browser-based visualization and patient monitoring interface.                                       |

## 34. Why These Technologies Were Selected

### 34.1 Python

Python provides a common language across the complete pipeline.

The same ecosystem can therefore be used for:

$$Data \rightarrow ML \rightarrow Backend \rightarrow Replay$$

This reduces integration complexity.

### 34.2 Pandas

The eICU data is tabular and large.

Pandas provides operations such as:

```
read_csv()
groupby()
median()
sort_values()
```

which are directly useful for physiological time-series preprocessing.

### 34.3 NumPy

NumPy is used for numerical arrays and operations such as:

$$z = \frac{x - \mu}{\sigma}$$

It also supports efficient manipulation of temporal model inputs.

### 34.4 PyTorch

PyTorch is responsible for the actual neural network.

It provides:

- tensors,
- automatic differentiation,
- neural-network modules,
- optimizers,
- GPU/CPU execution,
- model serialization.

### 34.5 scikit-learn

The model is not evaluated using accuracy alone.

The project calculates:

- ROC-AUC,
- PR-AUC,
- precision,
- recall,
- F1,
- confusion matrix.

These metrics are particularly useful when the positive class is imbalanced.

### 34.6 FastAPI

FastAPI provides the external interface to the system.

It allows:

- health checks,
- vital ingestion,
- patient state retrieval,
- alert retrieval,
- WebSocket communication.

### 34.7 Kafka

Kafka introduces an event-streaming layer.

Instead of:

$$Client \rightarrow Model$$

the architecture becomes:

$$Client \rightarrow API \rightarrow Kafka \rightarrow Consumer \rightarrow Model$$

This is more suitable for a streaming architecture.

### 34.8 Redis

The latest patient state is accessed frequently by the dashboard.

Redis provides fast key-value access:

$$state : 521456 \rightarrow current\ state$$

This avoids repeatedly reconstructing the latest state from historical database records.

### 34.9 PostgreSQL

A relational database is useful for durable structured records.

PostgreSQL stores:

- vitals,
- alerts,
- audit records.

This allows historical queries and auditability.

### 34.10 Docker Compose

Kafka, Redis, and PostgreSQL are infrastructure components.

Docker Compose allows these services to be started together with consistent configuration.

The project therefore has a reproducible local infrastructure environment.

## 35. Complete End-to-End Data Flow

The complete operation of CarePulse Twin can be summarized as follows.

### 35.1 Stage 1: Historical Data

The eICU dataset provides raw physiological observations.

$$\text{Raw eICU} \rightarrow \text{vitalPeriodic.csv.gz}$$

### 35.2 Stage 2: Preprocessing

The raw observations are grouped by patient and hour.

$$\text{Raw observations} \rightarrow \text{Hourly median values}$$

### 35.3 Stage 3: Sequence Construction

Hourly observations are converted into 24-hour windows.

$$139917 \text{ rows} \rightarrow 24 \times 6 \text{ sequences}$$

### 35.4 Stage 4: Training

The LSTM learns:

$$24 \text{ hours} \rightarrow 6 - \text{hour future deterioration}$$

### 35.5 Stage 5: Model Deployment

The trained model checkpoint is loaded by:

RiskEngine

### 35.6 Stage 6: Data Ingestion

A new observation enters:

$$\text{POST}/\text{api}/\text{v1}/\text{ingest}$$

### 35.7 Stage 7: Validation

The backend checks physiological ranges.

Invalid observations receive flags.

### 35.8 Stage 8: Kafka

Accepted observations are published to:

`carepulse.vitals`

### 35.9 Stage 9: Inference

The consumer receives the observation and updates the patient's 24-hour window.

$$W_{\text{patient}} \leftarrow W_{\text{patient}} \cup x_t$$

### 35.10 Stage 10: Model Prediction

Once 24 observations exist:

$$W_{\text{patient}} \rightarrow \text{LSTM} \rightarrow \text{risk}$$



### 35.11 Stage 11: Explainability

The system examines the latest vital values and recent trends.

$$\text{risk} + \text{physiological factors} \rightarrow \text{explanation}$$

### 35.12 Stage 12: State Storage

The latest state is stored in Redis.

Historical data and alerts are stored in PostgreSQL.

### 35.13 Stage 13: Dashboard

The frontend retrieves the state and displays:

- risk,
- severity,
- current vitals,
- recent trends,
- explanation.

## 36. Complete Demonstration Procedure

This section documents the commands used to demonstrate the working system.

### 36.1 Step 1: Enter the Project

```
cd ~/carepulse-twin-final
source .venv/bin/activate
```

### 36.2 Step 2: Start Infrastructure

Start the Docker services:

```
docker compose up -d
```

### 36.3 Step 3: Verify Kafka Topic

The topic is:

```
carepulse.vitals
```

It can be checked using:

```
docker exec carepulse-twin-final-kafka-1 \
/opt/kafka/bin/kafka-topics.sh \
--list \
--bootstrap-server localhost:9092
```

### 36.4 Step 4: Start Backend

```
python -m uvicorn backend.main:app \
--host 0.0.0.0 \
--port 8001
```

### 36.5 Step 5: Check Health

```
curl http://localhost:8001/health
```

Expected structure:

```
{
  "status": "ok",
  "model_available": true,
  "kafka_topic": "carepulse.vitals"
}
```

### 36.6 Step 6: Inspect Patients

For the current single-patient system:

```
curl http://localhost:8001/api/v1/patients/521456/state
```

### 36.7 Step 7: Replay Historical Patient

```
python -m replay.replay \
  --data data/processed/hourly_vitals.csv \
  --patient-id 521456 \
  --api http://localhost:8001 \
  --speed 3600
```

### 36.8 Step 8: Verify Redis State

```
docker compose exec redis \
redis-cli GET state:521456
```

### 36.9 Step 9: Verify PostgreSQL

```
docker compose exec postgres \
psql -U carepulse -d carepulse -c \
"SELECT COUNT(*) FROM vitals
WHERE patient_id = '521456';"
```

### 36.10 Step 10: Inspect Alerts

```
docker compose exec postgres \
psql -U carepulse -d carepulse -c \
"SELECT ts, severity, risk, message
FROM alerts
WHERE patient_id = '521456'
ORDER BY risk DESC
LIMIT 10;"
```

## 37. How to Explain the Demo

The recommended presentation sequence is:

1. Start with the architecture diagram.
2. Explain that historical eICU data is converted into hourly physiological observations.
3. Explain the six selected features.
4. Explain the 24-hour input window.
5. Explain the six-hour prediction horizon.
6. Explain that the LSTM produces a risk probability.
7. Explain that the backend receives new observations through FastAPI.
8. Show that the observation is published to Kafka.
9. Show the Kafka consumer processing the event.
10. Show Redis containing the latest state.
11. Show PostgreSQL containing persistent vitals and alerts.
12. Open the dashboard.
13. Show how the patient's risk changes during replay.
14. Show a HIGH or CRITICAL alert.
15. Explain the alert using the physiological factors displayed by the system.

## 38. Important Design Distinction

CarePulse Twin contains two different forms of reasoning.

### 38.1 Machine Learning Prediction

The LSTM answers:

Given the previous 24 hours of physiological data, how likely is a qualifying abnormality during the next six hours?

This is a learned temporal prediction.

### 38.2 Rule-Based Explanation

The explanation layer answers:

What observable physiological patterns are currently contributing to the elevated risk?

Examples include:

$$SpO_2 < 92$$

or:

$$RR > 24$$

or a recent increase in respiratory rate.

Therefore, the explanations are not claims that the LSTM itself explicitly reasons using those rules. They are a separate interpretability layer attached to the model output.

### 39. Current Limitations

The current implementation has several limitations.

1. The current model uses six selected physiological features.
2. The periodic dataset may contain missing measurements, particularly for variables that are not recorded continuously.
3. The current preprocessing representation is hourly.
4. The model predicts the defined research target rather than making a clinical diagnosis.
5. The explanation system is rule-based and is separate from the learned model.
6. The current demonstration uses historical replay to emulate real-time streaming.
7. The current frontend is primarily demonstrated around an individual patient.

These limitations define natural areas for future development.

### 40. Future Work

Possible extensions include:

1. Multi-patient command-center monitoring.
2. A unified dashboard showing all active patients.
3. True WebSocket-based multi-patient streaming.
4. More physiological variables.
5. Incorporation of aperiodic measurements.
6. Better missing-value handling.
7. Patient-specific baselines.
8. More advanced temporal models.
9. Model calibration.
10. Explainable AI methods such as SHAP or temporal attribution.
11. Model monitoring and drift detection.
12. Distributed Kafka consumers for higher throughput.
13. Cloud deployment.

### 41. Project Summary

CarePulse Twin transforms historical ICU physiological data into a real-time deterioration monitoring system.

The complete pipeline is:

eICU → Preprocessing → Hourly Vitals → 24h Sequences  
 → LSTM → FastAPI → Kafka → Risk Engine  
 → Redis/PostgreSQL → Dashboard

The most important design concept is that the system does not simply classify an individual vital measurement. Instead, it learns from a temporal physiological window:

$$24 \text{ hours} \times 6 \text{ features}$$

and predicts a future six-hour deterioration target.

The deployed system then combines this learned prediction with input validation, interpretable physiological factors, streaming infrastructure, persistent storage, and a real-time interface.

This creates an end-to-end architecture connecting:

|                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------|
| Data Science + Machine Learning + Backend Engineering<br>+Streaming Systems + Database Systems + Real-Time Monitoring |
|-----------------------------------------------------------------------------------------------------------------------|

which is the central engineering contribution of the CarePulse Twin project.

## A. Project Directory Structure

The relevant project structure is:

```
carepulse-twin-final/
|
+-- backend/
|   +-- main.py
|   +-- core.py
|
+-- data/
|   +-- eicu/
|       +-- vitalPeriodic.csv.gz
|       +-- vitalAperiodic.csv.gz
|       +-- patient.csv.gz
|   +-- processed/
|       +-- hourly_vitals.csv
|       +-- sequences.npz
|   +-- prepare.py
|
+-- ml/
|   +-- model.py
|   +-- train.py
|   +-- artifacts/
|       +-- carepulse_lstm.pt
|
+-- replay/
|   +-- replay.py
|
+-- frontend/
|   +-- index.html
|
+-- infra/
|
+-- tests/
|
+-- docker-compose.yml
+-- requirements.txt
+-- README.md
+-- VERIFICATION.md
+-- FINALSTATUS.md
```

## B. Key Commands

### B.1 Prepare Dataset

```
python -m data.prepare \
    --eicu-dir data/eicu \
    --out data/processed
```

### B.2 Train Model

```
python -m ml.train \
    --data data/processed/sequences.npz \
    --out ml/artifacts/carepulse_lstm.pt \
    --epochs 6 \
    --batch-size 256
```

### B.3 Start Backend

```
python -m uvicorn backend.main:app \
    --host 0.0.0.0 \
    --port 8001
```

### B.4 Replay Patient

```
python -m replay.replay \  
  --data data/processed/hourly_vitals.csv \  
  --patient-id 521456 \  
  --api http://localhost:8001 \  
  --speed 3600
```

### **B.5 Check Health**

```
curl http://localhost:8001/health
```

### **B.6 Check Current Patient State**

```
curl \  
http://localhost:8001/api/v1/patients/521456/state
```

### **B.7 Check Alerts**

```
curl \  
http://localhost:8001/api/v1/patients/521456/alerts
```